

BLOODBANK MANAGEMENT SYSTEM

A MINI-PROJECT REPORT

Submitted by

SHREYA K S 240701506

SHOBBIKA T 240701502

in partial fulfillment of the award of the degree

of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING



RAJALAKSHMI ENGINEERING COLLEGE, CHENNAI

An Autonomous Institute

CHENNAI

NOVEMBER 2025

BONAFIDE CERTIFICATE

Certified that this project **“BLOODBANK MANAGEMENT SYSTEM”** is the bonafide work of **“SHREYA K S, SHOBBIKA T”** who carried out the project work under my supervision.

SIGNATURE

Dr. V. JANANEE

ASSISTANT PROFESSOR SG

Dept. of Computer Science and Engg,
Rajalakshmi Engineering College
Chennai

This mini project report is submitted for the viva voce examination to be held on

INTERNAL EXAMINER

EXTERNAL EXAMINER

ABSTRACT

The **Blood Bank Inventory System** is designed to manage and track blood donations, donors, and requests efficiently.

Using **Java for the frontend** and **MySQL for the backend**, the system maintains real-time data of available blood stock, donor records, and hospital requests.

This project ensures faster processing, easy retrieval of information, and accurate inventory management for blood banks and hospitals.

ACKNOWLEDGEMENT

We express our sincere thanks to our beloved and honorable chairman **MR. S. MEGANATHAN** and the chairperson **DR. M.THANGAM MEGANATHAN** for their timely support and encouragement.

We are greatly indebted to our respected and honorable principal **Dr. S.N. MURUGESAN** for his able support and guidance.

No words of gratitude will suffice for the unquestioning support extended to us by our Head Of The Department **Dr. E.M. MALATHY** and our Deputy Head Of The Department **Dr. J. MANORANJINI** for being ever supporting force during our project work

We also extend our sincere and hearty thanks to our internal guide **Dr. V. JANANEE** , for her valuable guidance and motivation during the completion of this project.

Our sincere thanks to our family members, friends and other staff members of computer science engineering.

1.SHREYA K S

2. SHOBBIKA T

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO
	ABSTRACT	iv
1	INTRODUCTION	1
	1.1 INTRODUCTION	8
	1.2 SCOPE OF THE WORK	
	1.3 PROBLEM STATEMENT	8
	1.4 AIM AND OBJECTIVES OF THE PROJECT	8
2	SYSTEM SPECIFICATIONS	9
	2.1 HARDWARE SPECIFICATIONS	9
	2.2 SOFTWARE SPECIFICATIONS	9
3	MODULE DESCRIPTION	10
4	CODING	11
5	SCREENSHOTS	16
6	CONCLUSION AND FUTURE ENHANCEMENT	18
7	REFERENCES	24

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
5.1	INTRODUCTION PAGE	7
5.2	SCOPE AT WORK	15
5.3	WORKING CODE	16
5.4	SCREENSHOTS	16
5.5	CONCLUSION	17
5.6	REFERENCES	17

CHAPTER 1

INTRODUCTION

1.1 INTRODUCTION

The Blood Bank Inventory System automates the management of blood donations and requests.

It helps hospitals and donors keep track of availability in real time, eliminating manual errors and paperwork.

Built using Java and MySQL, it provides an efficient, user-friendly interface for managing blood data.

SCOPE OF THE WORK

- Manage donor and recipient records digitally
- Track available blood units by blood group.
- Automate donor registration and blood request processes.
- Generate reports for hospital and admin login validation.
- Provide secure admin login and data validation.

1.2 PROBLEM STATEMENT

Manual management of donor and blood records leads to delays, errors, and difficulty in tracking available blood units during emergencies. To overcome these issues, a computerized system using JavaSwing and MySQL is needed to efficiently store, update, and retrieve blood inventory and donor details in real time.

1.4 AIM AND OBJECTIVES OF THE PROJECT

The aim of this project is to develop a computerized Blood Bank Inventory System using Java and MySQL to manage blood donation and distribution efficiently. The main objectives are to maintain accurate donor and blood stock records, allow quick processing of blood requests, provide secure access for administrators, and ensure real-time information about blood availability for hospitals and users.

CHAPTER 2

SYSTEM SPECIFICATIONS

2.1 HARDWARE SPECIFICATIONS

Processor	:	Intel i5
Memory Size	:	8GB (Minimum)
HDD	:	1 TB (Minimum)

2.2 SOFTWARE SPECIFICATIONS

Operating System	:	WINDOWS 10
Front – End	:	JavaSwing
Back - End	:	MySql
Language	:	java,SQL

CHAPTER 3

MODULE DESCRIPTION

The Blood Bank Inventory System has two main modules: **Donor** and **Hospital**. It manage donor and hospital details, update blood stock, and handle blood requests. Donors can register, update their information, and see their donation history. Hospital can request blood, check its availability, and know the status of their request. These modules help the system work smoothly and manage all activities easily.

1. Donor login

When the person tries to login as a user then he/she will be prompted to enter the details of the donor in the table.

2. Hospital login

The hospital login allows hospitals to access the system securely, request required blood groups, and check the availability of blood units in the inventory.

CHAPTER 4

FRONT END CODE

```
package com.BloodVault;

import com.BloodVault.ui.UIHelper;

import com.BloodVault.donor.DonorLogin;

import com.BloodVault.donor.DonorSignup;

import com.BloodVault.hospital.HospitalLogin;

import com.BloodVault.hospital.HospitalSignup;


import javax.swing.*;

import java.awt.*;

/**
 * Entry point. Modern two-card home screen for Donor / Hospital.
 */

public class MainApp {

    public static void main(String[] args) {

        SwingUtilities.invokeLater(() -> {
```

```
JFrame frame = new JFrame("Blood Vault");

frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

frame.getContentPane().setBackground(UIHelper.BACKGROUND);

frame.setSize(760, 460);

frame.setLayout(new GridBagLayout());

frame.setResizable(false);


GridBagConstraints gbc = new GridBagConstraints();

gbc.insets = new Insets(16, 16, 16, 16);


// Title

JLabel title = new JLabel("Blood Vault");

title.setFont(UIHelper.TITLE);

title.setForeground(UIHelper.PRIMARY);

gbc.gridx = 0; gbc.gridy = 0; gbc.gridwidth = 2;

frame.add(title, gbc);


// Donor card

UIHelper.RoundedPanel donorCard = new UIHelper.RoundedPanel(new
GridBagLayout(), 14, UIHelper.CARD);
```

```
donorCard.setPreferredSize(new Dimension(300, 180));

GridBagConstraints d = new GridBagConstraints();

d.insets = new Insets(10, 10, 10, 10);

d.gridx = 0; d.gridy = 0; d.gridwidth = 2;

JLabel dTitle = new JLabel("Donor");

dTitle.setFont(UIHelper.SUBTITLE);

donorCard.add(dTitle, d);

d.gridy = 1; d.gridwidth = 1;

JButton dLogin = UIHelper.createPrimaryButton("Login");

dLogin.addActionListener(e -> {

    new DonorLogin();

    frame.dispose();

});

donorCard.add(dLogin, d);

d.gridx = 1;

JButton dSignup = UIHelper.createAccentButton("Sign Up");

dSignup.addActionListener(e -> {
```

```

        new DonorSignup();

        frame.dispose();

    });

    donorCard.add(dSignup, d);

    // Hospital card

    UIHelper.RoundedPanel hospCard = new UIHelper.RoundedPanel(new
GridBagLayout(), 14, UIHelper.CARD);

    hospCard.setPreferredSize(new Dimension(300, 180));

    GridBagConstraints h = new GridBagConstraints();

    h.insets = new Insets(10, 10, 10, 10);

    h.gridx = 0; h.gridy = 0; h.gridwidth = 2;

    JLabel hTitle = new JLabel("Hospital");

    hTitle.setFont(UIHelper.SUBTITLE);

    hospCard.add(hTitle, h);

    h.gridy = 1; h.gridwidth = 1;

    JButton hLogin = UIHelper.createPrimaryButton("Login");

    hLogin.addActionListener(e -> {

        new com.BloodVault.hospital.HospitalLogin();

        frame.dispose();

```

```
});

hospCard.add(hLogin, h);

h.gridx = 1;

JButton hSignup = UIHelper.createAccentButton("Sign Up");

hSignup.addActionListener(e -> {

    new HospitalSignup();

    frame.dispose();

});

hospCard.add(hSignup, h);

// Place cards side-by-side

gbc.gridy = 1; gbc.gridwidth = 1; gbc.gridx = 0;

frame.add(donorCard, gbc);

gbc.gridx = 1;

frame.add(hospCard, gbc);

frame.setLocationRelativeTo(null);

frame.setVisible(true);

});
```

DATABASE CREATION CODE

```
String id = Utils.generateDonorID();

// convert height and weight

Integer h = height.getText().trim().isEmpty() ? null :
Integer.valueOf(height.getText().trim());

Integer w = weight.getText().trim().isEmpty() ? null :
Integer.valueOf(weight.getText().trim());

boolean ok = new DonorDAO().insertDonor( id,

    fn.getText().trim(),

    ln.getText().trim(),

    Integer.parseInt(age.getText().trim()),

    bg.getSelectedItem().toString(),

    h, w,

    email.getText().trim(),

    mobile.getText().trim(),

    location.getText().trim(),

    p1 // this is your password

);
```

```
// show success or error message

if (ok) {

    JOptionPane.showMessageDialog(this, "Account created!\nYour Donor ID: " +
id, "Success", JOptionPane.INFORMATION_MESSAGE);

    new DonorDashboard();

    dispose();

} else {

    JOptionPane.showMessageDialog(this, "Database error — donor not saved.",
"Error", JOptionPane.ERROR_MESSAGE);

}

}
```

Backend code

```
package com.bloodvault.backend.db;

import java.sql.Connection;

import java.sql.DriverManager;

import java.sql.SQLException;

public class DB {

    private static final String URL =
"jdbc:mysql://localhost:3306/bloodvault?useSSL=false&serverTimezone=UTC";
```

```
private static final String USER = "root";    // your MySQL username

private static final String PASS = "Shobbi10*"; // your MySQL password


public static Connection getConnection() throws SQLException {

    return DriverManager.getConnection(URL, USER, PASS);

}

// Optional quick test

public static void main(String[] args) {

    try (Connection conn = getConnection()) {

        System.out.println("Connected");

    } catch (SQLException e) {

        e.printStackTrace();

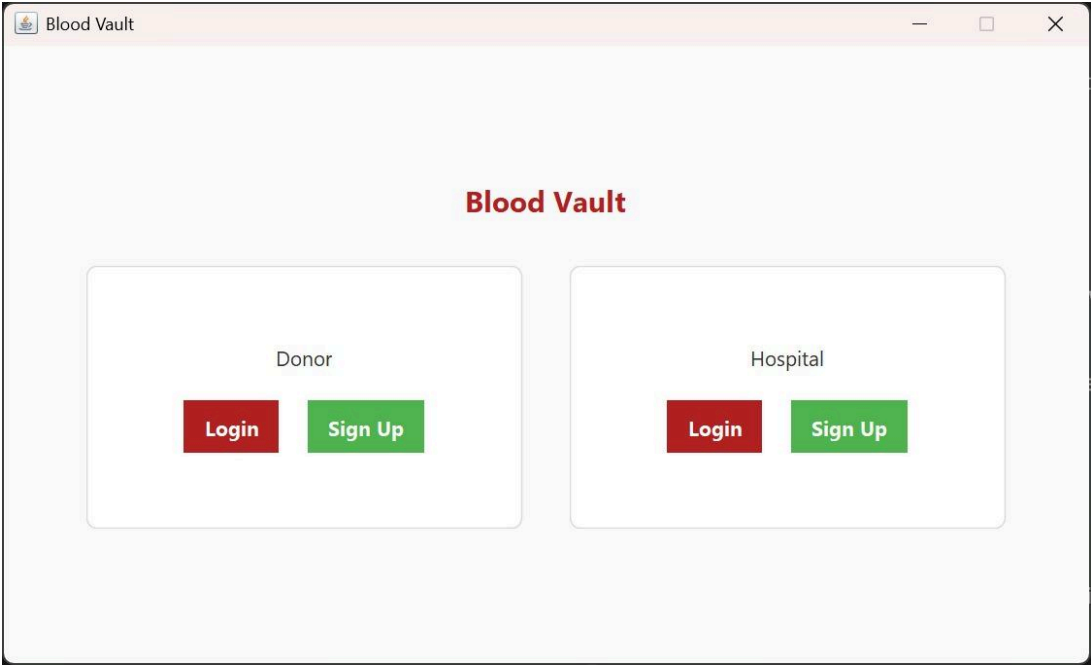
    }

}

}
```

CHAPTER 5

SCREEN SHOTS



Introduction page

Result Grid											
Filter Rows:											
Edit: Export/Import: Wrap Cell Content:											
donor_id	first_name	last_name	age	blood_group	height_cm	weight_kg	email	mobile	location	password_hash	created_at
D1001	Ram	S	22	A+			ram@gmail.com	12345678911	annanagar	b90f4a8368a6edea2143992200349448e142b2...	2025-10-28 20:44:05

Database creation

Donor Signup - Blood Vault

Donor Registration

First Name*:

Last Name*:

Age*:

Blood Group*:

A+

Height (cm):

Weight (kg):

Email*:

Mobile*:

Location*:

Password*:

Create Password

Confirm Password*:

Create Account

Donor login

Hospital Signup - Blood Vault

Hospital Registration

Hospital Name*:

License No*:

Location*:

Email*:

Phone*:

Password*: [Create Password](#)

Confirm Password*:

[Create Account](#)

Hospital login

Donor Dashboard - Blood Vault

Donor Dashboard

[Logout](#)

Donation History

Date	Hospital	Blood Group	Units	Status
2025-01-12	City Hospital	A+	1	Used
2025-03-20	Camp East	A+	1	Available

Health Details

Last Donation: 2025-03-20

Blood Count: 13.2 g/dL

BP: 120/80

Height / Weight: 170 cm / 65 kg

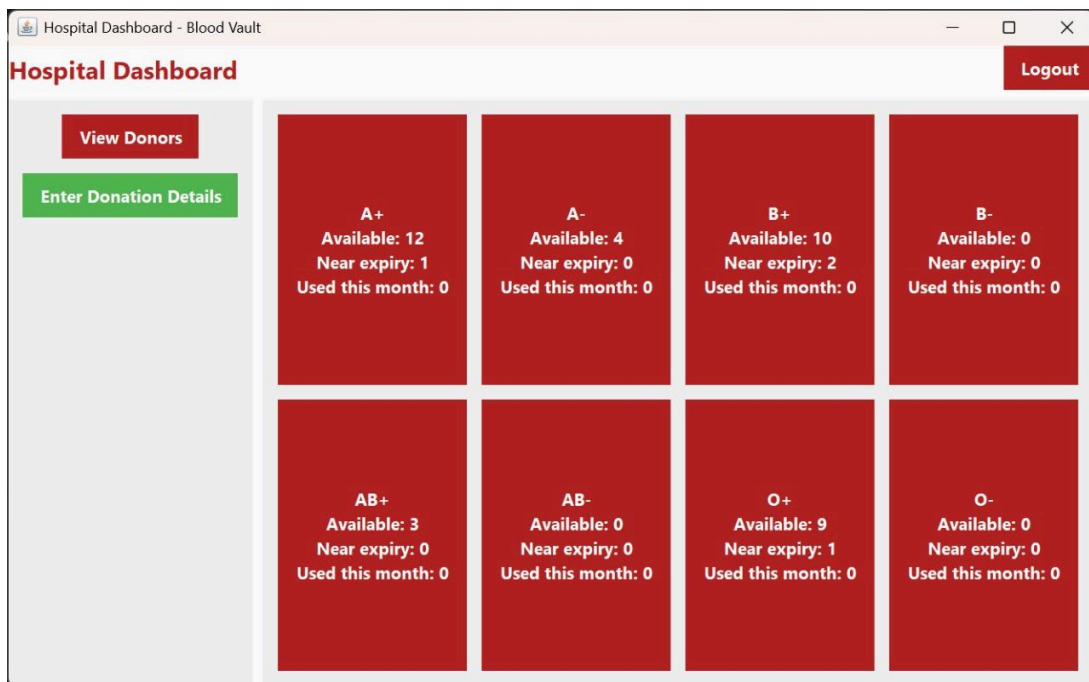
Heart Rate: 74 bpm

Awareness

Before donation: Hydrate, eat iron-rich meals, sleep well.

[Previous](#) [Next](#)

donor details



Hospital details

The screenshot shows the 'Hospital Dashboard - Blood Vault' interface with the 'Donor Data Entry - Blood Vault' form open. The form contains fields for Donor ID, Date (YYYY-MM-DD), Blood Group, Units, BP, and Heart Rate, followed by a 'Submit' button. The background shows the same dashboard as the previous image, but the 'Enter Donation Details' button is highlighted in green.

Enter Donation Details

Donor ID:

Date (YYYY-MM-DD):

Blood Group:

Units:

BP:

Heart Rate:

Submit

Donor entry details

CHAPTER 6

CONCLUSION AND FUTURE ENHANCEMENT

In the future, the system can be improved by adding SMS or email notifications to inform donors and hospitals about blood availability and donation camps. It can be connected to hospital management systems for easier access. A mobile app version can be developed for quick registration and emergency requests. The system can also include a QR code system for tracking each blood unit. Cloud storage can be used to keep data safe and accessible from anywhere. Advanced search and filtering options can be added for better results. Security can be enhanced with OTP-based logins. These features will make the system smarter and more user-friendly.

REFERENCES

1. <https://www.w3schools.com/java/>
2. <https://www.javatpoint.com/jdbc-tutorial>
3. <https://www.mysql.com/>
4. <https://www.geeksforgeeks.org/java-jdbc/>
5. <https://www.tutorialspoint.com/mysql/>
6. <https://www.javatpoint.com/jdbc-tutorial>
7. <https://www.mysql.com/>